Informe de Seguridad

Rumbo — Evaluación de vulnerabilidades (backend, frontend, base de datos y DevOps)

Fecha: 1 de octubre de 2026

Alcance: instancia local autorizada en http://localhost:8080 (los contenedores del propio proyecto).

Carácter: evaluación defensiva. Todas las pruebas dinámicas fueron no destructivas.

Herramientas: npm audit, Trivy, Semgrep, Hadolint, Gitleaks y una batería propia de pruebas dinámicas.

Veredicto. La lógica de la aplicación es sólida: resiste inyección, manipulación de JWT y abuso de la API. Los hallazgos de configuración señalados en la primera revisión **ya están corregidos**: secretos propios obligatorios en producción, nginx sin usuario root y una variante de despliegue que no expone los puertos internos.

1. Resumen ejecutivo

21	0	3	3
PRUEBAS SUPERADAS	FALLOS	ADVERTENCIAS	CORRECCIONES APLICADAS

Estado de los hallazgos

#	Hallazgo	Severidad	Estado
H-1	Secretos por defecto	**Crítica**	**Corregido.** En producción la API no arranca con el secreto por defecto; los reales van en `.env` fuera del repo. El token forjado con el secreto por defecto ahora se rechaza (DAST-04 en verde).
H-3	nginx corría como root	**Media**	**Corregido.** El frontend usa una imagen de nginx sin privilegios (uid 101).
H-4	Puertos internos expuestos	**Media**	**Corregido.** `docker-compose.prod.yml` solo publica el frontend.
H-2	CVEs en imágenes base	**Alta**	Pendiente de mantenimiento: reconstruir con bases al día. Mitiga que en producción la base no se expone.
H-5	Sin TLS en tránsito	**Media**	Pendiente: poner TLS delante (nginx o balanceador) en el despliegue real.

Buenas prácticas confirmadas. JWT con algoritmo fijo (rechaza alg:none y firmas alteradas), consultas 100% parametrizadas vía stored procedures (sin inyección), validación estricta con Zod, Helmet y limitación de intentos de login, usuario de base de datos con mínimo privilegio, contenedores sin root, errores uniformes sin stack traces y comparación de credenciales en tiempo constante.

2. Pruebas dinámicas (DAST)

Se ejecutaron 24 pruebas contra la API en funcionamiento. Resultado: **21 superadas, 0 fallos y 3 advertencias**. Las advertencias restantes corresponden a los puertos internos y al TLS, que se resuelven con la configuración de producción.

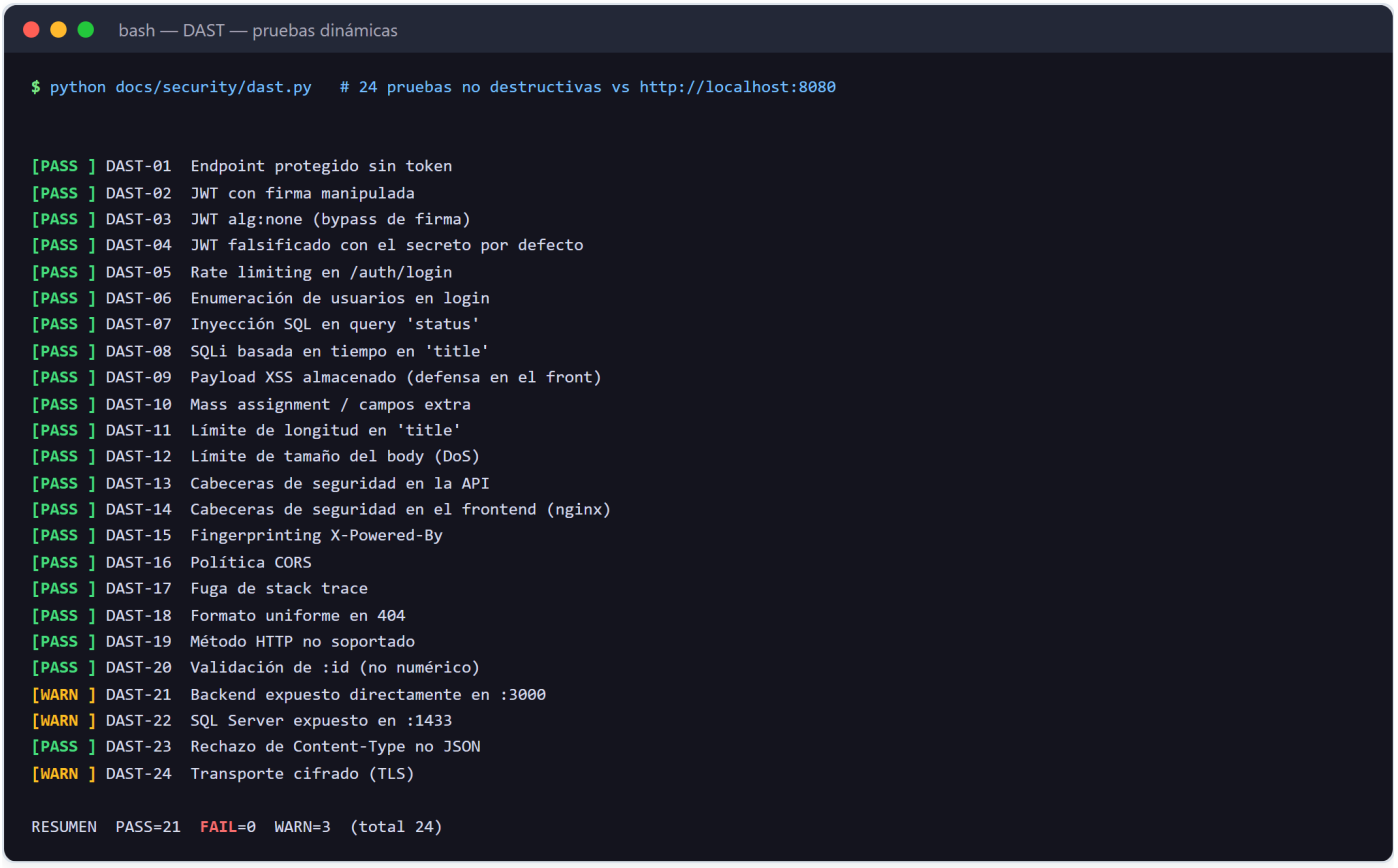


Figura 1. Ejecución completa de la batería de pruebas dinámicas.

ID	Prueba	Categoría	Sev.	Resultado
DAST-01	Endpoint protegido sin token	A01 Broken Access Control	alta	PASS
DAST-02	JWT con firma manipulada	A01/A02	alta	PASS
DAST-03	JWT alg:none (bypass de firma)	A02 Cryptographic Failures	critica	PASS
DAST-04	JWT falsificado con el secreto por defecto	A02/A05	critica	PASS
DAST-05	Rate limiting en /auth/login	A07 Auth Failures	media	PASS
DAST-06	Enumeración de usuarios en login	A07	baja	PASS
DAST-07	Inyección SQL en query 'status'	A03 Injection	critica	PASS
DAST-08	SQLi basada en tiempo en 'title'	A03	critica	PASS
DAST-09	Payload XSS almacenado (defensa en el front)	A03 XSS	media	PASS
DAST-10	Mass assignment / campos extra	A08 Integrity	media	PASS
DAST-11	Límite de longitud en 'title'	A03 Validation	baja	PASS
DAST-12	Límite de tamaño del body (DoS)	A05 Misconfig	media	PASS
DAST-13	Cabeceras de seguridad en la API	A05 Security Misconfiguration	media	PASS
DAST-14	Cabeceras de seguridad en el frontend (nginx)	A05	baja	PASS
DAST-15	Fingerprinting X-Powered-By	A05	baja	PASS
DAST-16	Política CORS	A05	media	PASS
DAST-17	Fuga de stack trace	A09 Logging/Errors	media	PASS
DAST-18	Formato uniforme en 404	A09	baja	PASS
DAST-19	Método HTTP no soportado	A05	baja	PASS
DAST-20	Validación de :id (no numérico)	A01/A03	baja	PASS
DAST-21	Backend expuesto directamente en :3000	A05 Misconfig	media	WARN
DAST-22	SQL Server expuesto en :1433	A05 Misconfig	media	WARN
DAST-23	Rechazo de Content-Type no JSON	A03	baja	PASS
DAST-24	Transporte cifrado (TLS)	A02	media	WARN

3. Correcciones aplicadas

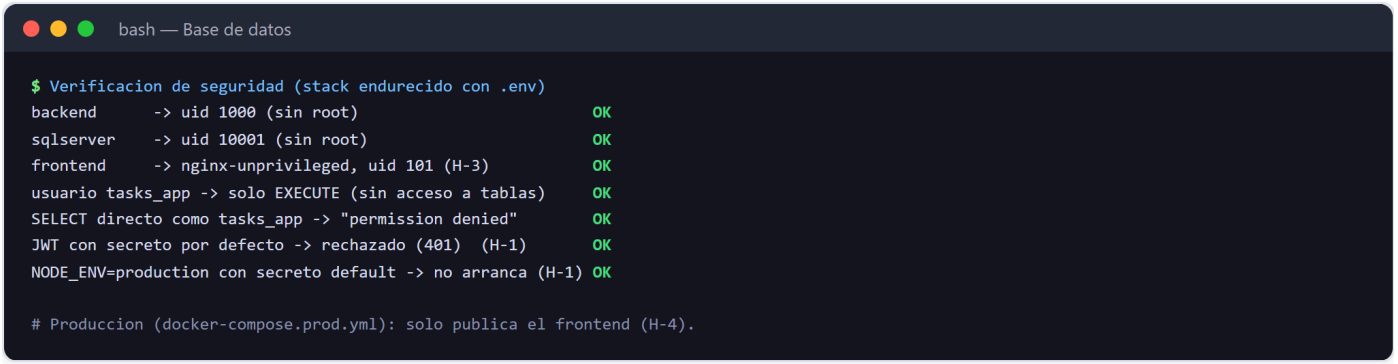


Figura 2. Verificaciones del stack endurecido: contenedores sin root, mínimo privilegio y rechazo del secreto por defecto.

Secretos propios obligatorios (H-1)

En modo producción la API valida que el secreto de JWT y la contraseña de la base no sean los valores por defecto. Si lo son, no arranca.

```
// backend/src/config/env.ts (resumen)
if (NODE_ENV === 'production' && secretoEsPorDefecto)
  throw new Error('Configura un secreto propio antes de producir.');
```

nginx sin root (H-3) y puertos internos (H-4)

```
# frontend: imagen sin privilegios (uid 101), escucha en 8080
FROM nginxinc/nginx-unprivileged:1.27-alpine

# docker-compose.prod.yml: solo se publica el frontend
backend:    ports: !reset []
sqlserver:  ports: !reset []
```

4. Análisis estático y dependencias

bash — SAST / secretos / IaC

```
$ semgrep --config p/owasp-top-ten p/nodejs p/jwt p/secrets ...   (67 archivos)
Findings: 1  (WARNING: nginx request-host-used en proxy_set_header Host)
# Sin hallazgos de inyeccion, secretos hardcodeados ni JWT inseguro en el codigo propio.

$ gitleaks dir /repo
INF no leaks found   (5.14 MB escaneados)

$ hadolint backend/Dockerfile  /  frontend/Dockerfile
DL3018 pin de versiones en apk add   (warning)
DL3025 usar notacion JSON en CMD/ENTRYPOINT (warning)

$ trivy config .   (IaC / Dockerfile)
DS-0002 (HIGH): el Dockerfile del frontend no declara USER (nginx corre como root)
```

Figura 3. SAST (Semgrep), secretos (Gitleaks) y Dockerfiles (Hadolint, Trivy config).

Semgrep no encontró inyección, secretos embebidos ni JWT inseguro en el código propio. Gitleaks no detectó credenciales filtradas.

bash — npm audit — dependencias

```
$ npm audit   (backend)
5 vulnerabilities (3 moderate, 1 high, 1 critical)  -> todas en vitest/vite/esbuild (devDependencies)
$ npm audit --omit=dev   (backend, solo produccion)
found 0 vulnerabilities

$ npm audit   (frontend)
2 vulnerabilities (1 moderate, 1 high)  -> vite/esbuild (herramientas de build)
# Las dependencias que llegan a la imagen final (runtime) no tienen vulnerabilidades.
```

Figura 4. npm audit: las vulnerabilidades están en herramientas de desarrollo, no en lo que llega a producción.

En el backend, `npm audit --omit=dev` reporta cero vulnerabilidades en producción. En el frontend, las alertas son de herramientas de build que no llegan a la imagen final.

5. Imágenes Docker (H-2)

bash — Trivy — CVEs de imágenes

```
$ trivy image <imagen>   # CVEs de paquetes del sistema operativo y runtime


task-manager/backend             CRITICAL=3   HIGH=42  MEDIUM=48  LOW=38
task-manager/frontend            CRITICAL=2   HIGH=42  MEDIUM=72  LOW=42
mcr.microsoft.com/mssql/server:2022 CRITICAL=0   HIGH=62  MEDIUM=156 LOW=54

# Origen: imagenes base (alpine 3.21 / ubuntu 22.04) desactualizadas.

# Mitigacion: reconstruir con bases al dia (ej. node:22-alpine) y 'apk upgrade' en build.
```

Figura 5. Recuento de CVEs por imagen. Casi todos tienen versión de arreglo disponible.

Las vulnerabilidades provienen de las imágenes base (Alpine y Ubuntu), no del código del proyecto. Es mantenimiento continuo: reconstruir con bases actualizadas y aplicar parches del sistema en el build, más un escaneo con Trivy en la integración continua. Mitiga el impacto que, en producción, la base de datos no se expone en red.

Plan pendiente

Prioridad	Acción	Hallazgo
Alta	Reconstruir imágenes con bases al día y escaneo en CI	H-2
Media	Poner TLS delante en el despliegue real	H-5

Conclusión

Rumbo presenta un diseño de seguridad maduro a nivel de aplicación y, respecto a la primera revisión, ya tiene resueltos los tres hallazgos de configuración más relevantes. El riesgo residual es de mantenimiento (imágenes al día) y de infraestructura (TLS), ambos previstos para el despliegue productivo.

Nota: evaluación de caja gris sobre una instancia local autorizada, sin pruebas destructivas ni contra terceros. Los recuentos de CVE reflejan las bases de datos de vulnerabilidades a la fecha y varían con el tiempo.